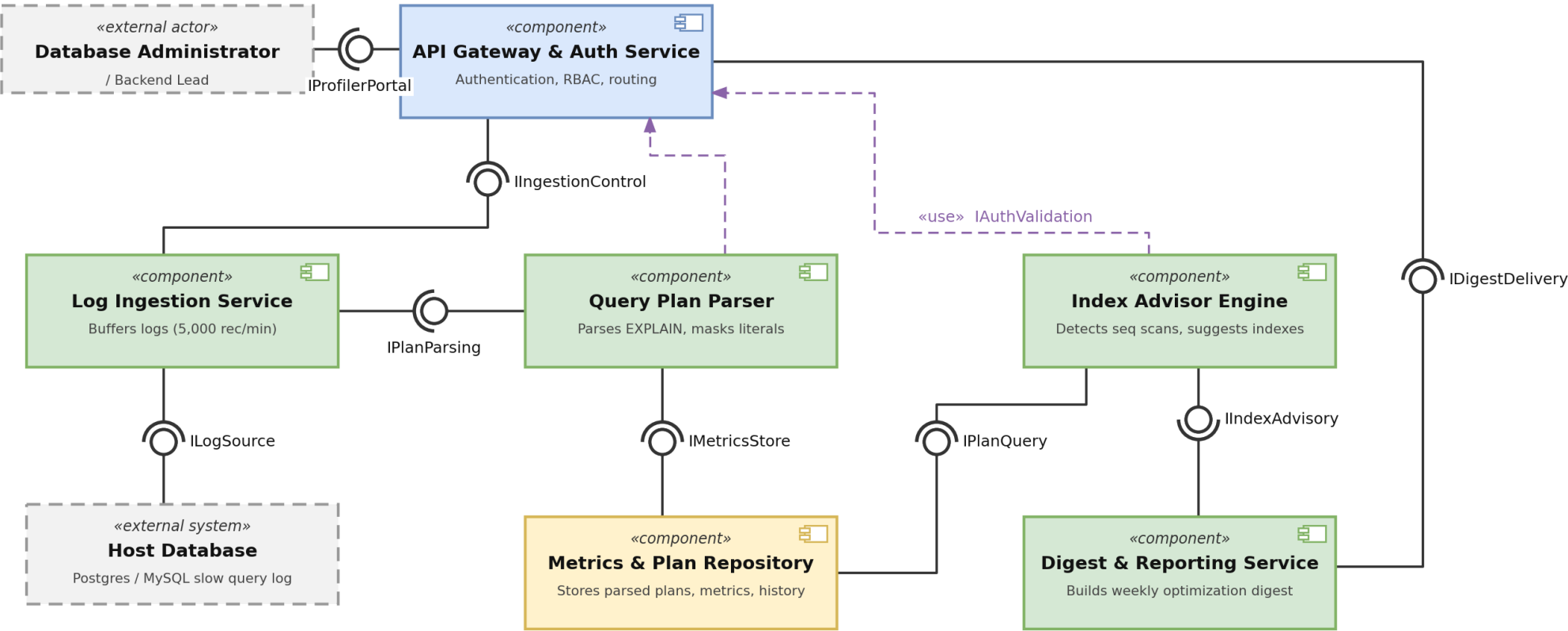


Database Query Performance Profiler — UML Component Diagram

Architectural Style: Microservices | Problem Statement #44 | Name: Aryaman D K | SRN: PES1UG24AM344



Lab 3 – Component Modelling & Architectural Pattern Selection

Problem Statement #44: Database Query Performance Profiler | Name: Aryaman D K | SRN: PES1UG24AM344

1. Architecture Selection

We chose the Microservices Architecture for the Database Query Performance Profiler System.

Style	Fit for this scenario	Verdict
Layered	Clean separation, but ingestion, parsing and weekly digest generation would sit in one deployable unit and scale together – wasteful, since only ingestion faces the 5,000 records/minute peak.	Rejected
Client–Server	Simple and centrally controlled, but one profiler server is a single point of failure; if it stalls, slow-query log entries are dropped and NFR-001 is violated.	Rejected
Microservices	Ingestion, parsing, index advising and digest generation have very different load profiles and release cycles, and each can be scaled, deployed and isolated on its own.	Selected

2. Two Reasons for the Choice

Reason 1 – Independent scaling of a very uneven workload. NFR-001 requires the tool to absorb up to 5,000 slow-query log records per minute without dropping entries or slowing the host database, whereas EXPLAIN-plan parsing is bursty and CPU-bound and the optimization digest runs only once a week. As separate services, additional instances of the Log Ingestion Service can be added at peak load alone, without duplicating the Query Plan Parser, the Index Advisor Engine or the Digest & Reporting Service. A single deployable unit would force the whole profiler to be over-provisioned for its noisiest component.

Reason 2 – Fault isolation and independent deployment per database engine. The Query Plan Parser and Index Advisor Engine must be updated whenever a Postgres or MySQL version changes its EXPLAIN output format. With microservices, that parser can be redeployed on its own while ingestion keeps buffering logs. Equally, a crash caused by a malformed or unsupported execution plan is contained inside the Index Advisor Engine and cannot halt log collection, so no slow-query entries are lost – exactly the failure mode NFR-001 forbids.

3. Security Advantage

The architecture exposes exactly one authenticated entry point: the **API Gateway & Auth Service**. The Database Administrator and Backend Lead reach the system only through the *IProfilerPortal* interface, and every internal call is token-checked through *IAuthValidation*, which also enforces role separation between the two actors. All other services sit on a private network with no public endpoint. Each service additionally holds a narrow, least-privilege credential – the Log Ingestion Service is granted read-only access to the host database's slow query log and never holds write credentials on production data. Because query text can contain sensitive literal values, the Query Plan Parser masks those literals before anything is written through *IMetricsStore*. The blast radius of a compromised service is therefore limited to that service's own data, which a single monolithic process could not guarantee.

4. Performance Benefit

Ingestion is decoupled from analysis. The Log Ingestion Service buffers incoming log records and hands them on asynchronously through *IPlanParsing*, so a slow parse or a long-running index analysis never blocks collection and never applies back-pressure to the host database – the key performance constraint in NFR-001. The expensive work (plan analysis and index recommendation) runs on its own horizontally scalable instances, and the Metrics & Plan Repository serves historical reads through *IPlanQuery* independently of the write path. The result is that ingestion latency stays flat under the 5,000 records/minute peak even while weekly digests are being generated.